FLASH HACK 2.0

Problem Statements

Design & Analysis of Algorithms × Operating Systems

Duration: 6 Hours	**Team Size:** 4 Members	**Level:** Intermediate	**Total Problems:** 12

> All problem statements require implementation of both DAA and OS concepts.Judging will be based on correctness of implementation, conceptual accuracy, and working demo.

PS 01

LifeLine: Hospital Emergency Triage Engine

Domain: Healthcare | Difficulty: **Beginner–Intermediate**

DAA: Sorting & Efficiency Analysis | OS: Process Scheduling

Background & Context

City General Hospital's emergency room receives dozens of patients simultaneously during peak hours. Currently, a nurse manually assigns priority — a slow, error-prone process that costs critical minutes. The hospital needs a system that instantly triages incoming patients and queues them for doctors in the most life-saving order.

Your Task

Build a patient triage system that assigns priority scores based on vitals (heart rate, BP, oxygen level, reported symptoms), sorts them using an efficient algorithm, and then simulates doctor allocation using a CPU scheduling strategy. The system should show at any point who is being treated, who is waiting, and estimated wait time for each patient.

Concepts You Must Implement

DAA Concepts	**Implement at least two sorting algorithms (e.g., Selection Sort for small batches, Merge Sort for large queues). Analyse and display their time complexity in context.**
OS Concepts	Model each patient as a 'process'. Implement Priority Scheduling and Round Robin to allocate doctors (CPUs). Show Gantt chart output with waiting time and turnaround time.
Tech Stack	Python / Java / C++ — your choice. No UI required; clear console or file output is fine.

Expected Deliverables

- Working triage sorter with at least 2 sorting algorithms and a comparison of their performance on the same input.
- Scheduler simulation showing patient queue, currently-being-treated, and wait time estimates.
- Sample input of at least 15 patients with varied vitals to demonstrate the system.

Bonus Challenge

- Add an 'aging' mechanism — patients waiting beyond a threshold automatically get their priority bumped up (prevents starvation).
- Allow dynamic patient arrival mid-simulation and show how the queue re-sorts in real time.

PS 02

CabGrid: Real-Time Cab Dispatch Optimizer

Domain: Transport & Logistics | Difficulty: **Beginner–Intermediate**

DAA: BFS/DFS & Brute Force | OS: Process Management & States

Background & Context

A cab aggregator startup is struggling with its dispatch logic. When a customer requests a ride, the system brute-forces through all available cabs — which works fine with 10 cabs but completely chokes at 500. They need a smarter traversal-based approach to find the nearest available cab quickly across a city modelled as a graph.

Your Task

Model the city as a graph where intersections are nodes and roads are edges. When a ride request comes in, use BFS to find the nearest available cab and DFS to explore possible pickup routes. Each cab in the system must go through defined process states: IDLE, DISPATCHED, EN_ROUTE, COMPLETED. The system should manage these state transitions correctly.

Concepts You Must Implement

DAA Concepts	**Implement BFS for nearest-cab search (shortest hops) and DFS for route exploration. Show clearly why BFS is preferred over brute force here with a comparison on a larger graph.**
OS Concepts	Model each cab as a process with states (New, Ready, Running, Waiting, Terminated). Implement state transition logic and show the state of all cabs at each dispatch event.
Tech Stack	Python / Java / C++. Graph can be hardcoded or loaded from an adjacency list file.

Expected Deliverables

- Graph-based city model with at least 20 nodes and BFS/DFS dispatch working correctly.
- Process state machine for cabs with all transitions demonstrated.
- Output log showing each ride request, which cab was dispatched, and state transitions triggered.

Bonus Challenge

- Add a brute force dispatcher alongside BFS and benchmark both on the same requests — show the time difference.
- Implement a surge pricing trigger when more than 60% of cabs are in DISPATCHED state.

DAA: Divide & Conquer / Merge Sort | OS: CPU Scheduling (SJF / FCFS)

Background & Context

SwiftShip, a fast-growing e-commerce platform, receives thousands of orders every hour. Their warehouse has multiple packing stations (workers), and they need to intelligently sort incoming orders by urgency and package them to shipping partners in an optimised sequence. Currently, orders pile up randomly, causing delays on express deliveries.

Your Task

Build an order fulfilment simulation. Incoming orders have attributes: order ID, weight, delivery type (express/standard), and time of arrival. Sort these orders using Merge Sort based on priority rules. Then assign them to packing stations (simulate as CPUs) using scheduling algorithms to minimise the time before the first express order ships.

Concepts You Must Implement

DAA Concepts	**Implement Merge Sort (Divide & Conquer) to sort orders. Prove correctness using Algorithm Correctness arguments — show that your merge step is invariant. Compare with a naive sort on a batch of 1000 orders.**
OS Concepts	Model packing stations as CPUs, orders as processes. Implement both FCFS and SJF. Show which strategy gets express orders out faster and why.
Tech Stack	Python / Java / C++. Generate a random dataset of 50–100 orders for demonstration.

Expected Deliverables

- Merge Sort implementation with step-by-step trace on a small sample.
- Scheduling simulation with FCFS vs SJF comparison showing average turnaround time.
- A summary report of which orders shipped first under each strategy.

Bonus Challenge

- Implement a hybrid scheduler: use SJF for express orders and FCFS for standard ones in parallel queues.

- Add a 'station breakdown' event mid-simulation and show how the system redistributes the load.

PS 04

CampusPath: Smart Course Prerequisite Planner

Domain: Education Technology | Difficulty: **Beginner–Intermediate**

DAA: Topological Sort / Decrease & Conquer | OS: Process Synchronization & Dependencies

Background & Context

A university's student portal has a broken course registration system. Students frequently register for advanced courses without completing prerequisites, then fail or get deregistered mid-semester. The academic office wants an automated system that validates registration, suggests a valid course completion order, and prevents dependency conflicts when multiple students register concurrently.

Your Task

Build a course dependency resolver. Given a curriculum graph (courses as nodes, prerequisites as directed edges), find a valid order for a student to complete all courses using Topological Sort. When multiple students register simultaneously, ensure no two students can corrupt the same registration record at the same time.

Concepts You Must Implement

DAA Concepts	**Implement Topological Sort using Decrease-and-Conquer (DFS-based). Detect and report cycles in the prerequisite graph (i.e., impossible course configurations). Handle at least 20 courses with complex dependencies.**
OS Concepts	Simulate concurrent student registrations as parallel processes. Use mutex locks / semaphores to protect shared course seat counters. Demonstrate what happens without synchronization (race condition) vs with it.
Tech Stack	Python (threading module) / Java (synchronized) / C++ (mutex). Graph loaded from a config file.

Expected Deliverables

- Topological sort producing valid course sequence for a given student's remaining courses.
- Cycle detection with a clear error message for impossible configurations.
- Concurrency demo: show a race condition, then fix it with proper synchronization.

Bonus Challenge

- Allow a student to specify courses they've already completed and generate a personalised remaining plan.
- Add a semester-by-semester layout — group topologically sorted courses into realistic semesters based on credit limits.

PS 05

CodeLens: Intelligent Code Review Assistant

Domain: Developer Tools | Difficulty: **Beginner–Intermediate**

DAA: String Matching (Horspool / Naive) | OS: File System & Access Methods

Background & Context

A small dev team is tired of manually reviewing code for banned patterns, copy-pasted logic, and style violations before every pull request. They want a lightweight CLI tool that scans their codebase, finds pattern matches across files, flags violations, and generates a structured review report — all without connecting to external services.

Your Task

Build a code review tool that takes a directory of source files and a list of banned patterns / search terms. It must scan files using an efficient string matching algorithm, detect matches with line numbers and context, and write a structured review report. The file access itself must follow a defined access strategy.

Concepts You Must Implement

DAA Concepts	**Implement the Horspool string matching algorithm. Compare its performance against naive pattern matching on a large file. Show the bad-character shift table construction.**
OS Concepts	Simulate file system access — implement sequential access for scanning and direct (random) access for jumping to flagged line numbers. Demonstrate both access methods working on the same file.

DAA Concepts	**Implement the Horspool string matching algorithm. Compare its performance against naive pattern matching on a large file. Show the bad-character shift table construction.**
Tech Stack	Python / C / C++. Works on .py, .java, or .c files. Output to a .txt or structured report.

Expected Deliverables

- Horspool matcher finding all occurrences of N patterns across M files with line numbers.
- Performance comparison: Horspool vs Naive on a file with 10,000+ lines.
- Structured review report grouped by file and violation type.
- Demonstration of sequential vs direct file access methods.

Bonus Challenge

- Implement multi-pattern matching — scan for all banned patterns in a single pass instead of N passes.
- Add a 'similarity score' between two files using character-level matching to detect copy-paste.

PS 06

FeedFlow: Personalised News Feed Curator

Domain: Social Media / Content | Difficulty: **Intermediate**

DAA: Space-Time Tradeoffs & Input Enhancement | OS: LRU Page Replacement

Background & Context

A news aggregation startup has a performance crisis. Their recommendation engine recomputes every user's personalised feed from scratch on every page load — taking 3–4 seconds per request. They need a smarter caching layer that stores recently computed feeds in a fixed memory budget, evicts the least relevant ones when full, and falls back to recomputation only when necessary.

Your Task

Simulate a news feed system where each user's feed is a computed ranked list of articles. Cache these computed feeds in a fixed-size store. When a new user requests a feed and the cache is full, evict using LRU policy. Show the cache hit/miss ratio. Also, use input

enhancement (preprocessing article metadata into a lookup table) to speed up ranking computation.

Concepts You Must Implement

DAA Concepts	**Implement a hash-based lookup table (input enhancement / space-time tradeoff) to pre-index articles by topic. Show that feed computation with the table is faster than without. Measure and display the tradeoff.**
OS Concepts	Implement LRU Page Replacement on your feed cache (fixed N slots = fixed memory frames). Track page faults (cache misses) and hits across a sequence of 50+ user requests. Show the reference string and frame states.
Tech Stack	Python / Java. Simulate articles and user preferences as data structures — no real web scraping needed.

Expected Deliverables

- Hash-based article index with measurable speedup over linear scan.
- LRU cache with correct eviction behaviour demonstrated on a reference string.
- Cache hit/miss statistics across a simulated workload of user requests.

Bonus Challenge

- Implement FIFO and Optimal page replacement alongside LRU and compare hit ratios on the same workload.
- Add a 'trending boost' — articles accessed by many users in the last window get priority in everyone's feed.

PS 07

TurnEngine: Competitive Turn-Based Game Validator

Domain: Gaming | Difficulty: **Intermediate**

DAA: Dynamic Programming (LCS / Edit Distance) | OS: Process Synchronization (Semaphores)

Background & Context

A small indie studio is building a turn-based strategy game. They have two problems: detecting if a player is repeating game states (stalling strategy) to declare a draw, and ensuring that when two players submit moves simultaneously on a shared game board, the board state never gets corrupted. They need a backend validator to handle both.

Your Task

Build the game backend that records board states after every move, uses LCS / Edit Distance to compare current board state with historical ones to detect repetition, and uses semaphores to ensure that move submission by Player 1 and Player 2 are mutually exclusive on the shared board resource.

Concepts You Must Implement

DAA Concepts	**Represent each board state as a string. Use LCS to measure similarity between the current state and all previous states. If similarity crosses a threshold (near-identical states), flag a potential draw. Use Edit Distance to measure how different two board states are.**
OS Concepts	Model each player's move submission as a process. The shared game board is a critical section. Use semaphores to enforce mutual exclusion. Demonstrate what happens without semaphores (board corruption) and with them.
Tech Stack	Python / Java / C++. Board can be a simple grid (e.g., chess-like or custom). No graphics needed.

Expected Deliverables

- LCS and Edit Distance working correctly on board state strings with threshold-based draw detection.
- Semaphore-protected board with a demonstrated race condition before and after the fix.
- A simulation of at least 20 moves showing state tracking and one draw detection trigger.

Bonus Challenge

- Track the sequence of moves as a 'move string' and use Edit Distance to compare two players' strategies — give a similarity report.
- Extend draw detection to the threefold repetition rule — exact repetition of position three times.

PS
08

NetWeave: City Fibre Optic Cable Planner

Domain: Smart City | Difficulty: **Intermediate**

DAA: Greedy (Activity Selection) | OS: Deadlock Detection (Banker's Algorithm)

Background & Context

A digital bank processes thousands of concurrent transactions daily. Two problems plague their system: transaction scheduling conflicts (two transfers wanting the same time slot on an account) and inter-account deadlocks (Account A waiting for Account B to release funds, while B waits for A — a classic circular wait). The core banking team needs both problems solved.

Your Task

Build a transaction management module. Use Greedy Activity Selection to schedule non-conflicting transactions on accounts (maximise number of transactions processed). Separately, simulate a multi-account system where concurrent transfers can create deadlocks — implement the Banker's Algorithm to check if a new transfer request leads to an unsafe state before approving it.

Concepts You Must Implement

DAA Concepts	**Implement Greedy Activity Selection to pick the maximum set of non-overlapping transactions on an account within a time window. Show that a greedy choice (pick earliest-finishing first) produces an optimal schedule.**
OS Concepts	Implement the Banker's Algorithm with allocation, max, and need matrices for accounts and their reserved funds. For every new transaction, run the safety algorithm. If unsafe, reject the transaction with an explanation.
Tech Stack	Python / Java / C++. Simulate at least 5 accounts and 20 transaction requests.

Expected Deliverables

- Activity selection scheduler showing maximum non-conflicting transactions for a given time window.

- Banker's Algorithm correctly identifying safe vs unsafe states for a set of concurrent transfers.
- A scenario demonstration where a deadlock is prevented by the Banker's rejection.

Bonus Challenge

- Implement deadlock detection (not just prevention) — given an already-deadlocked state, identify the circular wait chain.
- Add transaction rollback — when a request is rejected, suggest the smallest modification to make it safe.

PS 09

GridWatch: Power Grid Fault Detection System

Domain: Smart Infrastructure | Difficulty: **Intermediate**

DAA: Greedy (MST: Prim's & Kruskal's) | OS: Contiguous Memory Allocation

Background & Context

The city municipality wants to lay fibre optic cables to connect all neighbourhoods to a central internet exchange. Their budget is tight — they want the minimum total cable length that still connects every neighbourhood. Simultaneously, the network management software needs to allocate memory blocks to store routing tables for each neighbourhood, and they want to see the fragmentation impact of different allocation strategies.

Your Task

Model the city as a weighted graph (neighbourhoods as nodes, cable cost as edge weights). Find the Minimum Spanning Tree using both Prim's and Kruskal's algorithms and compare results. Separately, simulate memory allocation for routing tables using First Fit, Best Fit, and Worst Fit strategies on a fixed memory block.

Concepts You Must Implement

DAA Concepts	**Implement both Prim's and Kruskal's algorithms. Show step-by-step edge selection for both. Compare time taken on the same graph. Verify both produce the same total MST weight.**
OS Concepts	Simulate contiguous memory with a fixed size. Allocate and deallocate routing table blocks of varying sizes. Show internal/external

DAA Concepts	**Implement both Prim's and Kruskal's algorithms. Show step-by-step edge selection for both. Compare time taken on the same graph. Verify both produce the same total MST weight.**
	fragmentation for First Fit, Best Fit, and Worst Fit strategies on the same sequence of requests.
Tech Stack	Python / Java / C++. Graph loadable from a file. Memory simulation on a fixed array.

Expected Deliverables

- Prim's and Kruskal's producing MST with step-by-step trace and total cost.
- Memory allocation simulation with fragmentation statistics for all three strategies.
- A recommendation report: which MST algorithm is better for this graph type and why, which allocation strategy wastes less memory.

Bonus Challenge

- Add a budget constraint — if MST cost exceeds budget, find the maximum number of neighbourhoods connectable within budget (partial MST).
- Simulate memory compaction after fragmentation and show the memory map before and after.

PS 10

PacketPath: Network Service Boot Sequencer

Domain: Network Infrastructure | Difficulty: **Intermediate**

DAA: BFS / DFS & Brute Force | OS: Process Scheduling & Multithreading

Background & Context

A regional power utility manages a city-wide electricity grid modelled as a graph — substations are nodes and transmission lines are edges. When a fault is reported, engineers need to instantly identify all affected zones, find the shortest path to the fault from the nearest repair crew, and dispatch multiple repair teams in parallel without one team blocking another.

Your Task

Build a grid fault management system. Model the power grid as a graph. Use BFS to find the shortest path from the nearest repair depot to the faulty substation. Use DFS to map all zones

affected by the fault (reachable nodes from the fault point). Simulate multiple repair crews as threads/processes scheduled by the OS — each crew must be dispatched without conflicts.

Concepts You Must Implement

DAA Concepts	**Implement BFS for shortest path from depot to fault location. Implement DFS to find all affected zones reachable from the fault node. Show both traversals on the same graph and explain why each is the right tool for its respective task.**
OS Concepts	Model each repair crew as a thread/process. Implement Round Robin scheduling to dispatch crews fairly across multiple simultaneous faults. Show Gantt-style output of crew dispatch order with waiting time per crew.
Tech Stack	Python / Java / C++. Graph loadable from adjacency file. At least 20 nodes representing substations.

Expected Deliverables

- BFS shortest path from depot to fault substation with hop count and full path shown.
- DFS affected zone mapping — all nodes reachable from fault point listed.
- Round Robin crew dispatch simulation with Gantt output and waiting time per crew.
- Demo with at least 3 simultaneous faults showing correct multi-crew scheduling.

Bonus Challenge

- Add a brute force fault finder alongside BFS and show the performance difference on a 50+ node grid.
- Implement priority-based dispatch — faults at hospitals or critical nodes jump the scheduling queue.

PS 11

RosterSync: Hospital Staff Scheduling System

Domain: Healthcare Operations | Difficulty: **Intermediate**

DAA: Topological Sort / Decrease & Conquer | OS: Multithreaded Process Scheduling

Background & Context

A college network administrator is dealing with two problems. First, certain network services depend on others being initialised before they start — DNS must be up before the web proxy, the web proxy before the firewall, and so on. Booting them in the wrong order causes cascading failures. Second, bandwidth-heavy jobs (backups, OS updates, video streams) are hammering the network simultaneously with no coordination, starving smaller requests for minutes at a time.

Your Task

Build a network service boot sequencer and a bandwidth job scheduler. Use Topological Sort to determine the correct order to start all network services based on their dependency graph. Detect any circular dependencies that would make booting impossible. Then simulate bandwidth-heavy jobs as threads competing for the network — schedule them using preemptive priority scheduling to prevent starvation.

Concepts You Must Implement

DAA Concepts	**Implement Topological Sort (DFS-based Decrease & Conquer) on the service dependency graph. Detect cycles and report which services form the circular dependency. Handle at least 15 services with multi-level dependencies.**
OS Concepts	Model each bandwidth job as a thread with a priority and burst time. Implement preemptive priority scheduling. Demonstrate starvation of low-priority jobs without aging, then fix it with priority aging. Show a context switch log.
Tech Stack	Python / Java / C++. Service dependency graph from config file. At least 6 jobs with varying priorities and burst times.

Expected Deliverables

- Topological sort producing valid service boot order for a 15+ service dependency graph.
- Cycle detection with a clear error identifying which services form the circular dependency.
- Preemptive priority scheduling demo with context switch log.
- Starvation scenario and fix — low-priority job wait time before and after aging shown.

Bonus Challenge

- Add a critical service flag — if a critical service fails, all dependent services are skipped and flagged in output.
- Implement a time-of-day scheduling policy — job priorities shift automatically at night vs peak hours.

PS
12

BuildQueue: CI/CD Pipeline Job Optimiser

Domain: Developer Tools / DevOps | Difficulty: **Intermediate**

DAA: D&C, Greedy, DP crossover | OS: Scheduling, Synchronization, Deadlock

Background & Context

A college's resource management system handles three interconnected problems every semester: allocating classrooms to courses (a scheduling and optimisation problem), managing student access to shared GPU compute resources (a synchronisation problem), and sequencing the startup of all campus software services at the beginning of each semester (a dependency ordering problem). Currently all three are manual and error-prone.

Your Task

Build CampusCore — a unified resource manager with three modules.

- **Module 1: Room Allocation** — assign courses to rooms using Greedy interval scheduling to maximise room utilisation.
- **Module 2: GPU Cluster Access** — students submit compute jobs; use counting semaphores to limit concurrent access and Knapsack DP to select which jobs fit within GPU memory.
- **Module 3: Service Boot Sequencer** — use Topological Sort to find the correct startup order of campus software services from their dependency graph.

All three must run from a single main driver.

Concepts You Must Implement

DAA Concepts	**Module 1 uses Greedy interval scheduling (earliest-finish-time rule). Module 2 uses 0-1 Knapsack DP to select jobs within GPU memory budget. Module 3 uses Topological Sort (Decrease & Conquer) with cycle detection on the service dependency graph.**
OS Concepts	Module 2 uses a counting semaphore to limit concurrent GPU users to a defined maximum — blocks the (N+1)th user correctly. Module 3 simulates each service startup as a process — show scheduling of service initialisation with a Gantt-style output. Both must show synchronisation working correctly.

DAA Concepts	**Module 1 uses Greedy interval scheduling (earliest-finish-time rule). Module 2 uses 0-1 Knapsack DP to select jobs within GPU memory budget. Module 3 uses Topological Sort (Decrease & Conquer) with cycle detection on the service dependency graph.**
Tech Stack	Python / Java / C++. Each module as a separate class/file. Main driver runs all three on one semester's data.

Expected Deliverables

- Room allocation module: course-to-room Greedy assignment with room utilisation report.
- GPU module: Knapsack DP job selection + counting semaphore blocking (N+1)th user.
- Service boot module: valid Topological Sort order + cycle detection for bad configs.
- Unified main driver running all three modules on a shared semester dataset.

Bonus Challenge

- Cross-module dependency: if a student's GPU job is rejected in Module 2, their portal service in Module 3 is flagged as low priority.
- Semester-end analytics: room utilisation %, GPU queue wait times, and service boot success rate in one structured summary.

Judging Criteria

Criterion	Weight	Description
Algorithm Implementation	35%	Is the stated DAA concept correctly and completely implemented? Does the code reflect understanding, not just output?

Criterion	Weight	Description
OS Concept Integration	30%	Is the OS concept genuinely embedded in the solution or bolted on? Does it solve a real part of the problem?
Correctness & Working Demo	20%	Does the program run without errors on the given sample inputs? Are edge cases handled?
Code Quality & Documentation	10%	Is the code readable? Are functions named sensibly? Is there a brief README or comment explaining design choices?
Bonus / Innovation	5%	Did the team attempt the bonus challenge? Did they add something unexpected and useful?